DCIT 202- MOBILE APPLICATION DEVELOPMENT

Session 8 – Data Storage

Lecturer: Mr. Paul Nii Tackie Ammah, DCS
Contact Information: pammah@ug.edu.gh

UNIVERSITY OF GHANA
Department of Computer Science
School of Physical and Mathematical Sciences

Session Overview

- Goals and Objectives
  - Learn about data storage mechanisms in React Native.
  - Explore various storage mechanisms and understanding how to store, retrieve, and utilize the data.
  - Using SQLite for Local Storage

Session Outline

- Local Data
- Local Storage in React Native
- Using AsyncStorage
- Using Expo SecureStore
- Using Expo FileSystem
- Using SQLite for Local Storage
- Opening a Database Connection
- Creating a Table
- Inserting Data

Local Storage in React Native

- React Native applications often require a storage mechanism that persists data across application restarts or device reboots
- Local Storage refers to methods and tools used to store data locally on a device.
- This allows a mobile application to save data directly on the mobile device.
- Enabling the app to work offline or reduce dependence on a network connection for data retrieval.

Local Storage in React Native cont'd

- React Native framework does not include data storage APIs.
- Storage APIs are available as libraries
- Expo framework provides a few of such libraries to extend the capabilities of a React Native application
- Some of the storage options that work in React Native include:
  - AsyncStorage, SQLite, FileSystem, SecureStore, etc
- Local Storage enables **offline accessibility**, **performance improvements**, and **user preferences**

AsyncStorage

- AsyncStorage is a library that provides an ***asynchronous***, ***unencrypted***, ***persistent***, ***key-value*** storage API.
- **Async Storage** can only store string data.
- It is ideal for storing small amounts of data and simple flags or configuration settings.
- It has limited data structures, not ideal for complex data.
- To store object data, you need to serialize it first.
- For data that can be serialized to JSON, you can use **JSON.stringify()** when saving the data and **JSON.parse()** when loading the data.
- Installation (Expo App):

```
npx expo install @react-native-async-storage/async-storage
```

AsyncStorage Usage

- To use AsyncStorage, first import:

```
import AsyncStorage from '@react-native-async-storage/async-storage';
```

- Use **setItem(key, value)** both to add new data item and to modify existing item
- Use **getItem(key)** to retrieve stored item.
- **getItem(key)** returns a promise that either ***resolves to stored value when data is found for given key*** or ***returns null otherwise***.
- **removeItem(key, [callback])** removes item for a key, invokes (optional) callback once completed.
- **clear() r**emoves **whole** AsyncStorage data, for all clients, libraries, etc.
- For more info see: https://react-native-async-storage.github.io/async-storage/docs/api

AsyncStorage Example: Storing String

```
import AsyncStorage from '@react-native-async-storage/async-storage';


const storeData = async (key, value) => {
    try {
        await AsyncStorage.setItem(key, value);
    } catch (e) {
        // handle error
    }
};


await storeData('name', 'Peter');
```

AsyncStorage Example: Storing Object

```
import AsyncStorage from '@react-native-async-storage/async-storage';


const storeData = async (key, value) => {
    try {
                const jsonValue = JSON.stringify(value);
        await AsyncStorage.setItem(key, jsonValue);
    } catch (e) {
        // handle error
    }
};
const data = {'name': 'Patrick', 'age': 17, 'gender': 'male'};
await storeData("person", data);
```

AsyncStorage Example: Retrieving String

```
import AsyncStorage from '@react-native-async-storage/async-storage';


const getData = async (key) => {
    try {
        return await AsyncStorage.getItem(key);
    } catch (e) {
        // handle error
    }
};


const name = await getData("name");
```

AsyncStorage Example: Retrieving Object

```
import AsyncStorage from '@react-native-async-storage/async-storage';


const getData = async (key) => {
    try {
        const jsonValue = await AsyncStorage.getItem(key);
                return jsonValue != null JSON.parse(jsonValue) : null;
} catch (e) {
        // handle error
    }
};
const data = await getData('person');
```

Expo SecureStore

- A library that provides a way to encrypt and securely store key-value pairs locally on the device
- Supports both iOS and Android (Not compatible with devices running Android 5 or lower)
- Each Expo project has a separate storage system and has no access to the storage of other Expo projects.
- **Size limit for a value is 2048 bytes.**
  - **An attempt to store larger values may fail.**
- Installation: npx expo install expo-secure-store

Expo SecureStore

- On Android, values are stored in **SharedPreferences**, encrypted with **Android's Keystore system**.
- On iOS, values are stored using the **keychain service**
- For more info on configuration see: **https://docs.expo.dev/versions/latest/sdk/securestore/**

SecureStore Usage

```
import * as SecureStore from 'expo-secure-store';
//store data
async function save(key, value) {
        await SecureStore.setItemAsync(key, value);
}

//retrieve data
async function getValueFor(key) {
        return await SecureStore.getItemAsync(key);
}
```

Expo FileSystem

- Provides access to a file system stored locally on the device.
- It is also capable of uploading and downloading files from network URLs.
- The API takes **file://** URIs pointing to local files on the device to identify files.
- Each app only has read and write access to locations under the following directories:
  - **FileSystem.bundleDirectory:** URI to the directory where assets bundled with the application are stored. (read only access)
  - **FileSystem.documentDirectory**: URI pointing to the directory where user documents for this app will be stored.
  - **FileSystem.cacheDirectory**: URI pointing to the directory where temporary files used by this app will be stored. Files stored here may be automatically deleted by the system when low on storage
- Installation:

  npx expo install expo-file-system
- For more info see: https://docs.expo.dev/versions/latest/sdk/filesystem/

Expo FileSystem: Create Directory

```
import * as FileSystem from 'expo-file-system';

const photosDir = FileSystem.documentDirectory + 'photos/';

const dirInfo = await FileSystem.getInfoAsync(photosDir);

if (!dirInfo.exists) {
    console.log("Directory doesn't exist, creating…");
    //intermediates:If true, don't throw an error if file or directory doesn't exist at URI.
    await FileSystem.makeDirectoryAsync(photosDir, { intermediates: true });
}
```

Expo FileSystem: Create File and Write

```
import * as FileSystem from 'expo-file-system’;

const file = FileSystem.documentDirectory + ‘myfile.txt’;
Const contents = ‘Hello world’;

await FileSystem.writeAsStringAsync(file, contents, {
    encoding: FileSystem.EncodingType.UTF8,
})
```

Expo FileSystem: Read File

```
import * as FileSystem from 'expo-file-system’;

const file = FileSystem.documentDirectory + ‘myfile.txt’;

const contents = await FileSystem.readAsStringAsync(file, {
    encoding: FileSystem.EncodingType.UTF8,
})
```

FileSystem API

- **FileSystem.deleteAsync(fileUri, options):** Delete a file or directory. If the URI points to a directory, the directory and all its contents are recursively deleted.
- **FileSystem.downloadAsync(uri, fileUri, options):** Download the contents at a remote URI to a file in the app's file system.
- **FileSystem.getContentUriAsync(fileUri):** The local URI of the file. If there is no file at this URI, an exception will be thrown.
- **FileSystem.getInfoAsync(fileUri, options):** Get metadata information about a file, directory or external content/asset
- **FileSystem.makeDirectoryAsync(fileUri, options):** Create a new empty directory.
- **FileSystem.moveAsync(options):** Move a file or directory to a new location.

FileSystem API cont'd

- **FileSystem.readAsStringAsync(fileUri, options):** Read the entire contents of a file as a string
- **FileSystem.readDirectoryAsync(fileUri):** Enumerate the contents of a directory.
- **FileSystem.uploadAsync(url, fileUri, options):** Upload the contents of the file pointed by fileUri to the remote url.

- For a full list of available APIs see: **https://docs.expo.dev/versions/latest/sdk/filesystem/**

Using Expo SQLite for Local Storage

- SQLite provides a robust way to manage and query structured data locally.
- It's useful for applications that need to handle complex data or perform intricate queries offline.
- Expo SQLite library allows full access and usage to SQLite database from Expo Application.
- To use Expo SQLite in React Native (Expo Application), install the package using:

npx expo install expo-sqlite

Expo SQLite Usage

```
// Import the module from expo-sqlite.
import * as SQLite from 'expo-sqlite';

//create or open an existing database
const db = await SQLite.openDatabaseAsync('databaseName');
```

Creating Tables and Populating Data Using execAsync()

```
// `execAsync()` is useful for bulk queries when you want to execute
//  altogether.

await db.execAsync(`
PRAGMA journal_mode = WAL;
CREATE TABLE IF NOT EXISTS test (id INTEGER PRIMARY KEY NOT NULL, value TEXT NOT NULL, intValue INTEGER);
INSERT INTO test (value, intValue) VALUES ('test1', 123);
INSERT INTO test (value, intValue) VALUES ('test2', 456);
INSERT INTO test (value, intValue) VALUES ('test3', 789); `
);
```

Executing Write Operations with runAsync()

```
const result = await db.runAsync('INSERT INTO test (value, intValue) VALUES (?, ?)', 'aaa', 100);

// Binding unnamed parameters from arguments
await db.runAsync('UPDATE test SET intValue = ? WHERE value = ?', 999, 'aaa’);

// Binding unnamed parameters from array
await db.runAsync('UPDATE test SET intValue = ? WHERE value = ?', [999, 'aaa']);

// Binding named parameters from object
await db.runAsync('DELETE FROM test WHERE value = $value', { $value: 'aaa' });
```

Reading Data with getFirstAsync() and getAllAsync()

```
// `getFirstAsync()` is useful when you want to get a single row from
// the database.
const firstRow = await db.getFirstAsync('SELECT * FROM test');

// `getAllAsync()` is useful when you want to get all results as an array of objects.
const allRows = await db.getAllAsync('SELECT * FROM test');

for (const row of allRows) {
        console.log(row.id, row.value, row.intValue);
}
```

Using Prepared Statements

```
//create the prepared statment
const statement = await db.prepareAsync(
'INSERT INTO test (value, intValue) VALUES ($value, $intValue)'
);

//use the prepared statement
let result = await statement.executeAsync(
        { $value: 'bbb', $intValue: 101 }
);


console.log(result.lastInsertRowId, result.changes);
```

Expo SQLite APIs

- **deleteDatabaseAsync(databaseName):** Delete a database file.
- **openDatabaseAsync(databaseName, options):** Open a database.
- **getAllAsync():** Get all rows of the result set. This requires the SQLite cursor to be in its initial state.
- **getFirstAsync():** Get the first row of the result set.
- **resetAsync():** Reset the prepared statement cursor.
- **getEachAsync()**: This method fetches one row at a time from the database, potentially reducing memory usage compared to **getAllAsync()**
- **runAsync(), :** Executes a SQL query, returning information on the changes made. Ideal for SQL write operations such as INSERT, UPDATE, DELETE
- **finalizeAsync():** Finalize the prepared statement
- **executeAsync**(): Executes a SQL query, returning information on the changes made.

Assignment

- Find assignments in SAKAI